



INDIA.RUNS

Build what next India runs on

Team Name : Hawk-adarsh

Team Leader Name : Adarsh Prakash Singh

Problem Statement : Build an intelligent candidate discovery and ranking system to rank a 100,000-candidate pool for a Senior AI Engineer, Founding Team role, dynamically filtering consulting services companies and detecting 274 chronological honeypots.

Solution Overview

- **Proposed Solution:** An end-to-end Python ranking pipeline combining semantic experience curve scoring, weighted skill evaluation (Token Jaccard & Char 3-gram), and behavioral adjustments with strict chronological checks to filter synthetic profile anomalies.
- **Key Differentiators:** Matches terms contextually (e.g. scoring 'Vector DB' or 'ColBERT' under semantic search) rather than using static keyword lookup.
- **Honeypot Filter:** Zero-tolerance validator that flags 274 synthetic profiles.
- **Continuous Career Ratio:** Penalizes consulting-only/IT services experience continuously instead of using simple boolean filters.
- **Engagement Enveloping:** Applies a multiplicative multiplier to platform activity (notice period, response rates) to boost responsive talent.

JD Understanding & Candidate Evaluation

- **Key Requirements:** Senior AI Engineer with 5-9 years experience, expert in RAG, Vector Databases, Semantic Search, Fine-tuning, LLM Evaluation, and Python.
- **Candidate Signals:** Prioritizes active candidates (sub-30 day notice), high recruiter response rate, active platform engagement, and GitHub presence.
- **Beyond Keywords:** Custom Jaccard token similarity with a character-level 3-gram fallback to evaluate fit and capture related technologies dynamically.

Ranking Methodology

- **Retrieve & Score:** Iterates over the 100,000-candidate dataset, validating each profile for impossible chronologies. Profiles with conflicts receive a hard penalty of -999.0.
- **Scoring Formula:** $\text{Fit} = \text{Relevance}(\text{Skills}) + \text{Relevance}(\text{Title}) + \text{Relevance}(\text{Exp}) + \text{Location}$.
Selected roles like Applied ML and Backend are boosted, and consulting experience is penalized.
- **Signal Combination:** Modifies the base fit score by a continuous career penalty $(1 - \text{ConsultingRatio})^2$ and scales by a behavior factor $[0.5, 1.2]$ based on activity.

Explainability & Data Validation

- **Explainability:** Generates custom, fact-based 1-2 sentence descriptions summarizing experience, skill proficiency, location, and key platform activity.
- **Hallucination Prevention:** Avoids LLM-based reasoning generation at inference time; reasons are constructed programmatically from verified profile fields.
- **Data Validation (274 Honeypots):** Catches startup founding year violations (250), calendar mismatches (16), and expert skills zero-duration conflicts (8).

End-to-End Workflow

- **Step 1:** Parse candidates from candidates.jsonl.
- **Step 2:** Run validation rules to flag and filter out 274 chronological honeypots.
- **Step 3:** Calculate base scores for skills, title, experience curve, and location.
- **Step 4:** Scale scores using consulting penalties and behavioral engagement signals.
- **Step 5:** Sort descending, slice the top 100, and generate explainability sentences.
- **Step 6:** Write to submission.csv and run format validation checks.

System Architecture

- **Candidate Streamer:** Memory-efficient parser reading candidates.jsonl.
- **Validation Engine:** Hard rules checking startup founding years, calendar months, and zero-duration skill claims.
- **Relevance Scorer:** Calculates experience fit (5-9 years optimal), title semantic boost, and skill matching.
- **Modifier Engine:** Computes consulting ratio penalty and adjusts scores via active notice period, recruiter response, and platform activity.
- **Output Generator:** Generates explainability text, formats submission.csv, and checks compliance.

Results & Performance

- **Ranking Quality:** The top 100 candidates contain 0% honeypots, have a median experience of 6.2 years, and are highly qualified in core AI fields.
- **Performance:** Scores 100,000 candidates on CPU in 19.6 seconds (<150MB RAM). Extremely fast due to token similarity caching.

What technologies, frameworks, and tools were used and why were they selected for this solution?

- **Core Runtime:** Python 3.8+ (ensures low overhead and fast startup).
- **Libraries:** Standard libraries (json, csv, math, datetime, re) for zero external dependencies during runtime.
- **Verification & Testing:** Jupyter & nbconvert for scoring analysis, and python-pptx for automated reporting.

Submission Assets

- **GitHub Repository:** <https://github.com/adarsh3908/India-Runs.git>
- **Metadata File:** submission/submission_metadata.yaml
- **Submission CSV:** submission/submission.csv
- **HuggingFace Space:** <https://huggingface.co/spaces/adarsh3908/redrob-ranker>
- **Presentation Deck:** report/approach_presentation.pptx



INDIA.RUNS

Build what next India runs on

THANK YOU